Vigenère-Squad : une équipe d'agents IA qui casse les chiffrements classiques et explique sa démarche

Rapport technique — version 1.0.0

Dawid Rozanski — M1 Cybersécurité, SUP DE VINCI

Novembre 2026

Table des matières

1 Résumé **2**

2 Abstract **2**

3 Introduction **2**

4 Rappels de cryptanalyse **3**
4.1 Indice de coïncidence . . . 3
4.2 Kasiski et Friedman . . . 3
4.3 Attaque du χ^2 et escalade . . . 3
4.4 Score de quadrigrammes et recuit simulé . . . 3
4.5 Autoclave . . . 3
4.6 Distance d'unicité de Shannon . . . 4

5 Architecture du système **4**
5.1 Agents et liste fermée d'outils . . . 4
5.2 Protocole d'appel d'outils . . . 5
5.3 Boucle de vérification, réessais et replis . . . 5
5.4 Référence sans modèle (configuration a) . . . 5

6 Protocole expérimental **5**
6.1 Corpus . . . 5
6.2 Configurations comparées (ablation) . . . 6
6.3 Métriques . . . 6
6.4 Matériel et environnement . . . 7

7 Résultats **7**
7.1 Étude d'ablation . . . 7
7.2 Taux de succès par procédé et longueur . . . 7
7.2.1 Configuration a_baseline . . . 7
7.2.2 Configuration b_qwen3_4b . . . 8
7.2.3 Configuration c_qwen3_1.7b . . . 8
7.3 Longueur minimale de succès par procédé . . . 8
7.4 Lien avec Shannon . . . 8
7.5 Robustesse, défaillances et injection de consignes . . . 9
7.6 Explications et exercices . . . 9

8 Analyse critique **9**
8.1 Apport mesuré des agents . . . 9

8.2 Effet de la taille du modèle 11
8.3 Limites 12

9 Modèle de menace **12**

10 Science ouverte et reproductibilité **12**

11 Conclusion **13**

12 Références **13**

13 Annexe : interface en ligne de commande **13**

1 Résumé

Ce travail présente Vigenère-Squad, un système multi-agents exécutable sur un ordinateur portable sans GPU qui reçoit un texte chiffré par un procédé classique (César, affine, Vigenère, autoclave, substitution monoalphabétique, Beaufort), identifie le procédé, retrouve la clé et le clair, puis rédige une explication pédagogique et un exercice vérifié. Cinq agents spécialisés (Classifieur, Cryptanalyste, Linguiste, Pédagogue, Vérificateur), dotés chacun d'une liste fermée d'outils, sont coordonnés par un orchestrateur explicite avec une boucle de vérification. Le modèle de langage local (Qwen3 via Ollama) décide et explique ; toute la cryptanalyse (indice de coïncidence, Kasiski, Friedman, χ^2, recuit simulé, attaque de l'autoclave) est réalisée par du code déterministe testé. Une étude d'ablation sur 180 chiffrés compare une référence sans modèle à deux tailles de modèle ; elle est complétée par des expériences de robustesse, de défaillances injectées, d'injection de consignes et par une comparaison avec la distance d'unicité de Shannon. Tout est reproductible par `make reproduce` et archivé avec DOI.

2 Abstract

This report presents Vigenère-Squad, a multi-agent system that runs on a GPU-less laptop. Given a ciphertext produced by a classical cipher (Caesar, affine, Vigenère, autokey, monoalphabetic substitution, Beaufort), it identifies the cipher, recovers the key and the plaintext, then writes a step-by-step explanation together with a tool-verified exercise. Five specialised agents with closed tool lists are coordinated by an explicit orchestrator with a verification loop. The local language model (Qwen3 served by Ollama) decides and explains, while all cryptanalytic computation is done by tested deterministic code. An ablation study on 180 ciphertexts compares a no-LLM baseline with two model sizes; robustness, fault-injection, prompt-injection experiments and a comparison with Shannon's unicity distance complete the evaluation. Everything is reproducible with `make reproduce` and archived with a DOI.

3 Introduction

Au XIXe siècle, Babbage puis Kasiski ont cassé le chiffre de Vigenère par une démarche méthodique : repérer les répétitions, estimer la longueur de la clé, ramener le problème à plusieurs chiffrements de César attaqués par analyse de fréquences. Cette démarche se découpe en rôles (identifier, attaquer, valider, expliquer), ce qui en fait un banc d'essai naturel pour un système multi-agents.

La question de recherche est double : (1) un petit modèle de langage local, cantonné à la décision et à l'explication, apporte-t-il un gain mesurable par rapport à une heuristique codée en dur qui enchaîne les mêmes outils ? (2) à quel coût (temps, défaillances) et avec quelles garanties de sécurité ?

Contributions : une boîte à outils de cryptanalyse classique testée (couverture > 70 %) ; une architecture à cinq agents avec liste fermée d'outils imposée au contrôle et au décodage ; un protocole expérimental à graines fixes avec étude d'ablation ; une chaîne de publication auditable (traces JSONL, résultats signés, archive Zenodo).

4 Rappels de cryptanalyse

4.1 Indice de coïncidence

L'indice de coïncidence d'un texte de n lettres, d'effectifs n_i, est $IC = \sum_i n_i(n_i - 1)/(n(n - 1))$. Il vaut environ 0,078 en français et $1/26 \approx 0{,}038$ pour un texte aléatoire. Une substitution monoalphabétique conserve l'IC ; un chiffre polyalphabétique le rapproche de la valeur aléatoire. C'est le premier critère du Classifieur.

4.2 Kasiski et Friedman

Dans un Vigenère, deux occurrences d'un même fragment de clair chiffrées avec la même portion de clé produisent des répétitions dont les distances sont multiples de la longueur L de la clé. L'outil `kasiski` recense les répétitions de trigrammes, compte les diviseurs des distances et privilégie le plus grand diviseur dont l'effectif reste proche du maximum (pour ne pas retenir systématiquement 2 ou 3). L'outil `friedman` calcule l'estimation de Friedman $L \approx (\kappa_p - \kappa_r)\, n/((n-1)\, IC - \kappa_r n + \kappa_p)$ et, plus robuste sur texte court, l'IC moyen des colonnes pour chaque longueur candidate ; la plus petite longueur dont l'IC moyen atteint 90 % du maximum observé est retenue (ce seuil relatif écarte les sous-multiples favorisés par une clé à lettres répétées).

4.3 Attaque du χ^2 et escalade

Une fois L fixée, chaque colonne est un César : le décalage minimisant la statistique du χ^2 entre fréquences observées et fréquences du français est retenu. La clé est ensuite affinée par escalade : chaque lettre est remplacée par celle qui maximise le score de quadrigrammes du texte entier.

4.4 Score de quadrigrammes et recuit simulé

Un modèle de langue par quadrigrammes (log-probabilités, plancher pour les quadrigrammes absents) est appris sur trois romans du domaine public (Zola, Hugo), disjoints des textes utilisés pour les chiffrés. Le score normalisé vaut 0 pour un texte aléatoire et 1 pour un texte français typique ; c'est la mesure de plausibilité du Linguiste. Pour la substitution quelconque ($26! \approx 2^{88}$ clés), un recuit simulé échange deux lettres de la permutation et accepte une dégradation avec une probabilité décroissante, avec redémarrages à graine fixe.

4.5 Autoclave

Dans l'autoclave (clé auto), la clé est l'amorce suivie du clair lui-même : $c_i = p_i + k_i$ avec $k_i = p_{i-L}$ pour $i \geq L$. Le flux de clé ne se répète pas, ce qui prive Kasiski de son fondement. En revanche, pour une longueur d'amorce L donnée, la colonne j (positions $j, j + L, j + 2L, \ldots$) ne dépend que de la lettre d'amorce K_j : $p_j = c_j - K_j$, puis $p_{j+rL} = c_{j+rL} - p_{j+(r-1)L}$. Chaque colonne est donc attaquée séparément sur 26 hypothèses par χ^2, puis la clé est affinée par escalade, et la meilleure longueur est retenue. L'expérience de la section « Résultats » montre l'échec de Kasiski sur l'autoclave :

procédé	bonne longueur en tête (%)	dans le top 3 (%)	répétitions de trigrammes	IC global	IC moyen des colonnes à la vraie longueur
autoclave	13.3	30.0	40.7	0.041	0.0426
vigenere	63.3	70.0	97.7	0.0439	0.0754

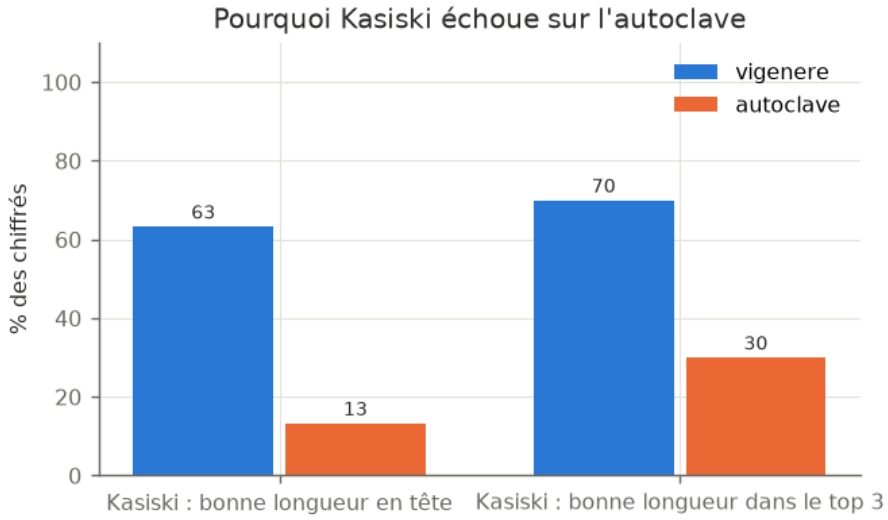


Figure 1: Kasiski : Vigenère contre autoclave

4.6 Distance d'unicité de Shannon

Shannon définit la distance d'unicité $U = H(K)/D$, où $H(K)$ est l'entropie de la clé et D la redondance de la langue par lettre. Pour un Vigenère à clé aléatoire de longueur L, $H(K) = L \log_2 26 \approx 4{,}70\, L$ bits. La redondance est estimée à partir du modèle de quadrigrammes : $D = \log_2 26 - H_4$ où H_4 est l'entropie conditionnelle $H(X_4|X_1X_2X_3)$ = 2.623 bits, d'où D = 2.077 bits ; on compare aussi à la valeur classique $D \approx 3.4$ bits (entropie du français ≈ 1,3 bit par lettre).

5 Architecture du système

5.1 Agents et liste fermée d'outils

Agent	Rôle	Outils autorisés	Sortie
Classifieur	mono/poly, ordre des hypothèses	`indice_coincidence`, `frequences`	`famille`, `procedes_probables`, `justification`
Cryptanalyste	choisit et paramètre l'attaque, désigne le résultat	`kasiski`, `friedman`, `chi2_cesar`, `force_brute_affine`, `attaque_vigenere`, `attaque_beaufort`, `attaque_autoclave`, `recuit_substitution`	`id_resultat`, `procede`, `justification`
Linguiste	juge la plausibilité, renvoie en arrière	`score_quadrigrammes`	`plausible`, `justification`, `procede_suivant`
Pédagogue	explication pas à pas, exercice (nouvelle clé)	aucun (lit les traces)	`explication`, `exercice`

Agent	Rôle	Outils autorisés	Sortie
Vérificateur	vérifie le corrigé de l'exercice	`chiffrer_exercice`, `casser_exercice`	`valide`, `commentaire`

Les cinq agents partagent un même modèle chargé en mémoire ; ils se distinguent par leur prompt système (fichier versionné, version et empreinte SHA-256 enregistrées dans chaque trace), leurs outils et leur schéma de sortie Pydantic.

5.2 Protocole d'appel d'outils

À chaque tour, le modèle renvoie un objet JSON `{"pensee", "appels", "conclusion"}` dont le schéma est transmis à Ollama (décodage contraint). Si `appels` n'est pas vide, l'orchestrateur : (1) refuse tout outil hors de la liste de l'agent ; (2) valide les arguments (Pydantic, champs inconnus interdits, bornes) ; (3) exécute l'outil avec un délai maximal ; (4) renvoie un résumé compact. Sinon, la conclusion est validée par le schéma de l'agent.

Deux choix limitent ce que le modèle peut altérer : le texte chiffré n'est jamais un argument (il est injecté depuis l'état partagé), et le clair annoncé est toujours celui d'un résultat d'outil désigné par identifiant (`R1`, `R2`…). Le modèle ne peut donc ni modifier le chiffré, ni fabriquer un clair.

5.3 Boucle de vérification, réessais et replis

Le Cryptanalyste propose un résultat, le Linguiste le juge ; s'il n'est pas plausible, le procédé est rejeté et une nouvelle hypothèse est tentée (au plus 6 itérations, une par procédé). Un procédé déjà rejeté ne peut pas être reproposé : la trace d'un premier essai avec `qwen3:0.6b` montrait un Cryptanalyste relançant quatre fois César et épuisant la boucle. Un **garde-fou** déterministe refuse un « plausible » dont le score normalisé est inférieur à 0,65. Ce seuil a été mesuré sur le corpus : les bons clairs obtiennent au moins 0,70 (texte de 80 lettres) et les clairs erronés produits par une mauvaise attaque au plus 0,62 (80 lettres) et 0,48 (150 lettres) ; un premier seuil à 0,45 laissait passer des clairs erronés lorsque le Linguiste les déclarait plausibles. Chaque appel au modèle bénéficie de deux réessais (réponse vide, JSON invalide, conclusion hors schéma, délai dépassé) ; au-delà, l'orchestrateur se replie sur une décision déterministe (classement par IC, attaque suivante, seuil de plausibilité, explication par gabarit) et le consigne. Si aucune hypothèse n'est acceptée, le meilleur score est retenu (statut `meilleur_effort`).

5.4 Référence sans modèle (configuration a)

L'heuristique codée en dur classe le chiffré par IC (seuil 0,060), essaie les attaques dans un ordre fixe (César, affine, substitution puis Vigenère, autoclave, Beaufort, ou l'inverse), s'arrête au premier clair dont le score normalisé atteint 0,70 et retient sinon le meilleur score. Elle utilise exactement les mêmes outils que les agents.

6 Protocole expérimental

6.1 Corpus

Le script `data/generer_corpus.py --seed 42` produit 180 chiffrés par variante : 6 procédés × 6 longueurs (80, 150, 300, 600, 1 000, 2 000 lettres) × 5. Les extraits commencent en début de phrase dans quatre œuvres du domaine public jamais utilisées pour l'apprentissage du modèle de langue (Verne ×2, Flaubert, Voltaire). Clés : César 1–25, affine avec $a \neq 1$ premier avec 26, permutation aléatoire pour la substitution, mots aléatoires de 3 à 12 lettres pour les polyalphabétiques. Les textes sources sont épinglés par SHA-256.

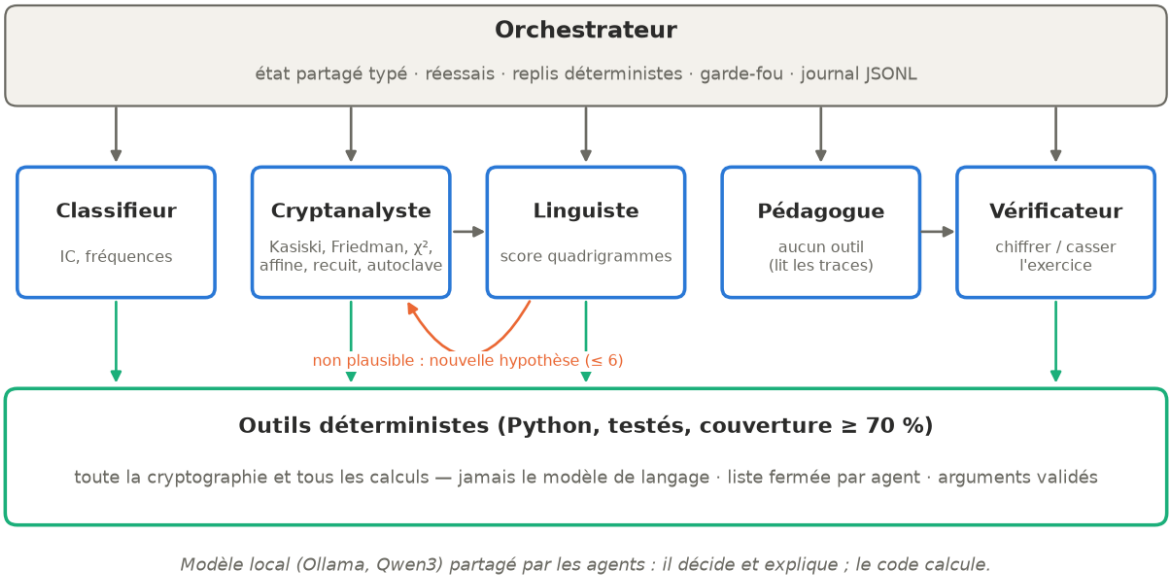


Figure 2: Architecture

Variantes de robustesse : `sans_espaces` (lettres seules), `fautes5` (5 % des lettres du clair remplacées au hasard avant chiffrement), `anglais` (*Frankenstein*), évaluée avec le modèle de langue français puis anglais.

6.2 Configurations comparées (ablation)

Configuration	Description
a_baseline	outils + heuristique codée en dur, sans modèle de langage
b_qwen3_4b	système multi-agents, Qwen3 4B (≈ 3 Go, exécutable sur 8 Go)
c_qwen3_1.7b	système multi-agents, Qwen3 1,7B (taille différente)

La référence est évaluée sur les 180 chiffrés ; les agents, dont le coût est plus élevé, sur un sous-ensemble stratifié de 12 chiffrés (un par procédé, aux longueurs 150 et 1 000 lettres ; 6 chiffrés de 600 lettres pour la robustesse « sans espaces » et les défaillances, 3 pour l'injection de consignes). Ce dimensionnement découle de la vitesse mesurée sur la machine de l'auteur (≈ 20 s par appel au modèle, ≈ 6 appels par chiffré) et du délai de remise ; la référence couvre les trois variantes de robustesse sur les 180 chiffrés. Pour une comparaison appariée, la référence est aussi rapportée sur ce même sous-ensemble. Chaque mesure est répétée 3 fois (graine du modèle `42 × 1000 + répétition`, température 0,3) et rapportée en moyenne ± écart type.

6.3 Métriques

Taux de clair exact (toutes les lettres justes), par procédé et par tranche de longueur ; longueur minimale de succès à 90 % ; taux de procédé correct ; précision par lettre ; itérations moyennes de la boucle ; temps et mémoire par chiffré ; taux de repli ; appels refusés ; garde-fous déclenchés ; validité des exercices ; qualité des explications (grille sur 10, 10 explications tirées au hasard).

6.4 Matériel et environnement

Matériel : x86_64 (4 cœurs), 15.6 Go, Linux-7.0.0-1017-azure-x86_64-with-glibc2.39. Ollama 0.40.16. Modèles et empreintes : `qwen3:1.7b` `8f68893c685c`, `qwen3:4b` `359d7dd4bcda`. Durée totale de `make reproduce` : 185.9 (17 serveurs GitHub Actions en parallèle, 1131.9 min de calcul cumulé, exécution 42829992108) min ; mémoire : 75.5 Mo pour Python ; modèles chargés par Ollama : qwen3:1.7b 1.3 Go, qwen3:4b 2.3 Go.

7 Résultats

7.1 Étude d'ablation

configuration	chiffrés	clair exact (%)	réf. (a) même sous-ensemble (%)	procédé correct (%)	lettres justes (%)	itérations	durée (s)	mémoire (Mo)	replis (%)	appels refusés
a_baseline	180	93.7 ± 0.3	–	99.4	98.0 ± 0.2	2.0	0.4	76	0.0	0
b_qwen3_4b	12	97.2 ± 4.8	100.0	86.1 ± 4.8	97.4 ± 4.5	2.6 ± 0.8	550.7 ± 145.8	63 + 42	61.1 ± 9.6	0
c_qwen3_1.7b	72	88.9 ± 4.8	100.0	86.1 ± 17.4	90.1 ± 4.6	4.4 ± 0.2	444.1 ± 18.8	63 + 42	100.0	0

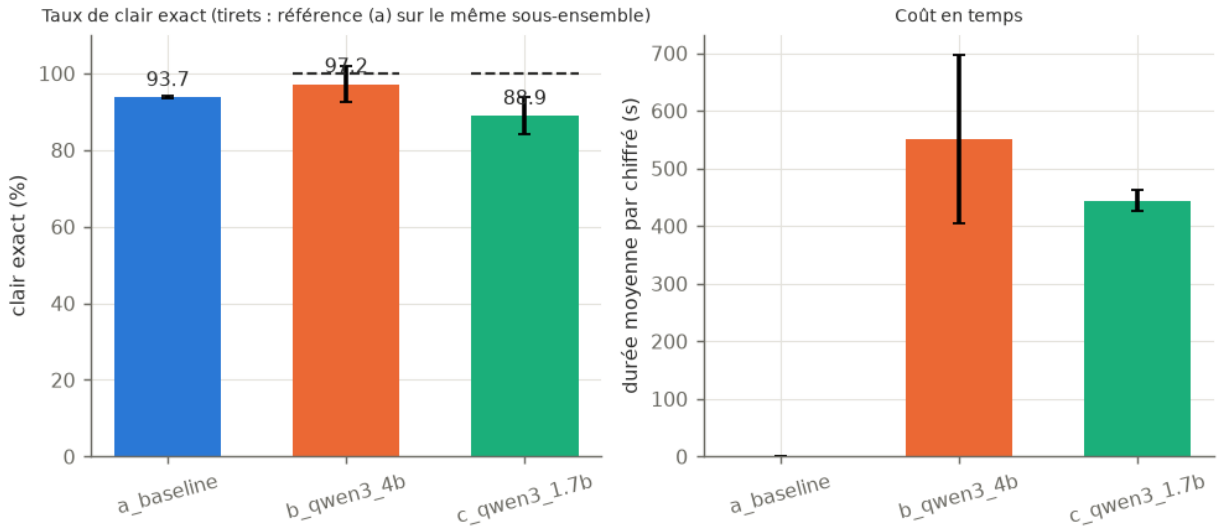


Figure 3: Ablation

Apport mesuré des agents :

- **b_qwen3_4b** : 97.2 % de clairs exacts contre 100.0 % pour la référence (a) sur les mêmes 12 chiffrés, soit un écart de **-2.8 points** ; 2.64 itérations en moyenne contre 2.03 ; durée multipliée par 1232 ; replis déterministes dans 61.1 % des exécutions.
- **c_qwen3_1.7b** : 88.9 % de clairs exacts contre 100.0 % pour la référence (a) sur les mêmes 12 chiffrés, soit un écart de **-11.1 points** ; 4.39 itérations en moyenne contre 2.03 ; durée multipliée par 993 ; replis déterministes dans 100.0 % des exécutions.

7.2 Taux de succès par procédé et longueur

7.2.1 Configuration a_baseline

procédé	080-119	120-249	250-499	500-899	900-1499	1500+	longueur min. 90 %
cesar	100.0	100.0	100.0	100.0	100.0	100.0	080-119
affine	100.0	100.0	100.0	100.0	100.0	100.0	080-119
vigenere	60.0	100.0	100.0	100.0	100.0	100.0	120-249
autoclave	80.0	100.0	100.0	100.0	100.0	100.0	120-249
substitution	0.0	60.0	73.3 ± 11.6	100.0	100.0	100.0	500-899
beaufort	100.0	100.0	100.0	100.0	100.0	100.0	080-119

7.2.2 Configuration b_qwen3_4b

procédé	080-119	120-249	250-499	500-899	900-1499	1500+	longueur min. 90 %
cesar	–	100.0	–	–	100.0	–	120-249
affine	–	100.0	–	–	100.0	–	120-249
vigenere	–	100.0	–	–	100.0	–	120-249
autoclave	–	100.0	–	–	100.0	–	120-249
substitution	–	100.0	–	–	100.0	–	120-249
beaufort	–	66.7 ± 57.7	–	–	100.0	–	900-1499

7.2.3 Configuration c_qwen3_1.7b

procédé	080-119	120-249	250-499	500-899	900-1499	1500+	longueur min. 90 %
cesar	–	100.0	–	–	100.0	–	120-249
affine	–	100.0	–	–	100.0	–	120-249
vigenere	–	100.0	–	–	66.7 ± 57.7	–	non atteinte
autoclave	–	100.0	–	–	100.0	–	120-249
substitution	–	100.0	–	–	100.0	–	120-249
beaufort	–	66.7 ± 57.7	–	–	33.3 ± 57.7	–	non atteinte

7.3 Longueur minimale de succès par procédé

Mesurée sur une grille fine (20 à 2 000 lettres, 20 tirages par point pour l’attaque adaptée au procédé connu, 10 pour la référence (a) qui doit aussi identifier le procédé) ; c’est la plus petite longueur à partir de laquelle le taux de clair exact reste au moins égal à 90 %.

procédé	min. 90 % — procédé connu (lettres)	min. 90 % — référence (a), procédé inconnu (lettres)
affine	20	40
autoclave	150	80
beaufort	80	80
cesar	20	60
substitution	> max	> max
vigenere	80	80

7.4 Lien avec Shannon

longueur de clé	min. 90 % (longueur inconnue)	min. 90 % (longueur connue)	U (D estimée)	U (D = 3,4 bits)
3	60	20	6.8	4.1
4	40	30	9.1	5.5
5	60	40	11.3	6.9
6	60	60	13.6	8.3
8	80	80	18.1	11.1

longueur de clé	min. 90 % (longueur inconnue)	min. 90 % (longueur connue)	U (D estimée)	U (D = 3,4 bits)
10	100	100	22.6	13.8
12	150	100	27.2	16.6

La longueur minimale de succès mesurée (longueur de clé connue) vaut entre 2.9 à 4.4 fois la distance d'unicité estimée. La distance d'unicité est une borne d'information : au-delà, un seul clair est *possible* en principe, mais elle ne dit rien de l'effort de calcul. Une attaque statistique colonne par colonne a besoin de bien plus de texte, car chaque colonne ne contient que n/L lettres et le χ^2 y est bruité. L'écart entre « longueur connue » et « longueur inconnue » mesure le coût de l'estimation de L (Kasiski et Friedman), qui devient dominant pour les clés longues.

7.5 Robustesse, défaillances et injection de consignes

expérience	configuration	chiffrés	clair exact (%)	lettres justes (%)	replis (%)	erreurs LLM	garde-fous
defaillances	b_qwen3_4b	6	88.9 ± 9.6	94.8 ± 8.9	77.8 ± 9.6	106	0
injection	b_qwen3_4b	3	88.9 ± 19.2	94.4 ± 9.6	88.9 ± 19.2	12	14
rob. anglais / modèle en	a_baseline	180	93.9 ± 0.6	98.0 ± 0.4	0.0	0	0
rob. anglais / modèle fr	a_baseline	180	37.6 ± 0.3	69.7 ± 1.5	0.0	0	0
rob. fautes5	a_baseline	180	85.2 ± 0.3	97.9 ± 0.0	0.0	0	0
rob. sans_espaces	a_baseline	180	94.3 ± 0.6	98.7 ± 0.3	0.0	0	0
rob. sans_espaces	b_qwen3_4b	6	77.8 ± 9.6	94.9 ± 8.8	61.1 ± 9.6	6	0

7.6 Explications et exercices


```
{
  "b_qwen3_4b": {
    "exercices_valides_pct": 70.0,
    "n_explications": 10,
    "taux_exact_echantillon": 100.0
  }
}
```


8 Analyse critique

8.1 Apport mesuré des agents

Pour **b_qwen3_4b**, l'écart avec la référence (a) sur les mêmes chiffrés est de -2.8 points (une perte), inférieur ou égal à deux écarts types (9.6 points) : il n'est pas significatif au sens retenu, la référence fait aussi bien sur ce problème. Les agents utilisent 2.64 itérations en moyenne, se replient sur une décision déterministe dans 61.1 % des exécutions, et 0 appel(s) d'outil hors liste ont été bloqués.

Le procédé le plus difficile pour b_qwen3_4b est **beaufort** (83.3 % de clairs exacts).

Pour **c_qwen3_1.7b**, l'écart avec la référence (a) sur les mêmes chiffrés est de -11.1 points (une perte), supérieur à deux écarts types (9.6 points) : l'écart est significatif au sens retenu. Les agents utilisent 4.39 itérations en moyenne, se replient sur une décision déterministe dans 100.0 % des exécutions, et 0 appel(s) d'outil hors liste ont été bloqués.

Le procédé le plus difficile pour c_qwen3_1.7b est **beaufort** (50.0 % de clairs exacts).

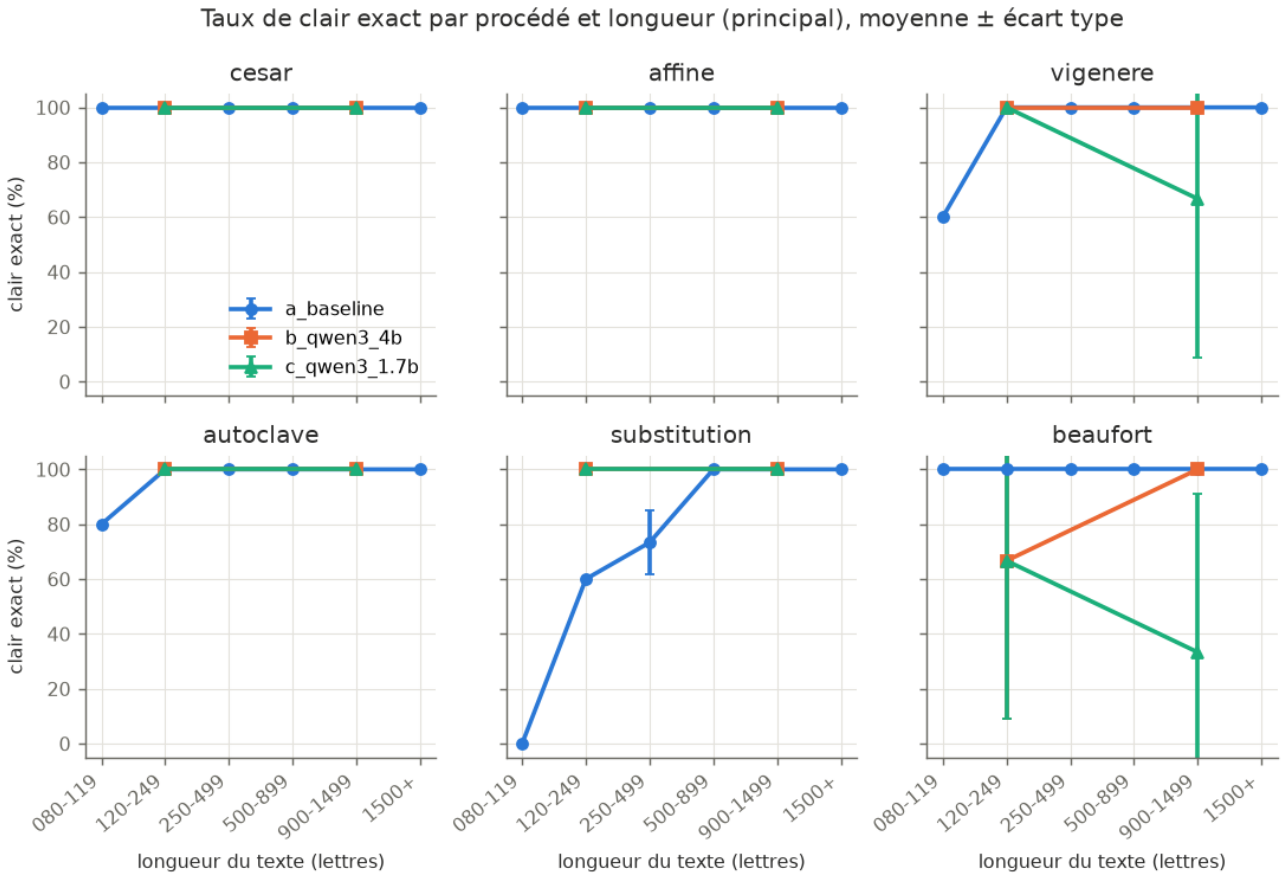


Figure 4: Taux de clair exact par procédé et longueur

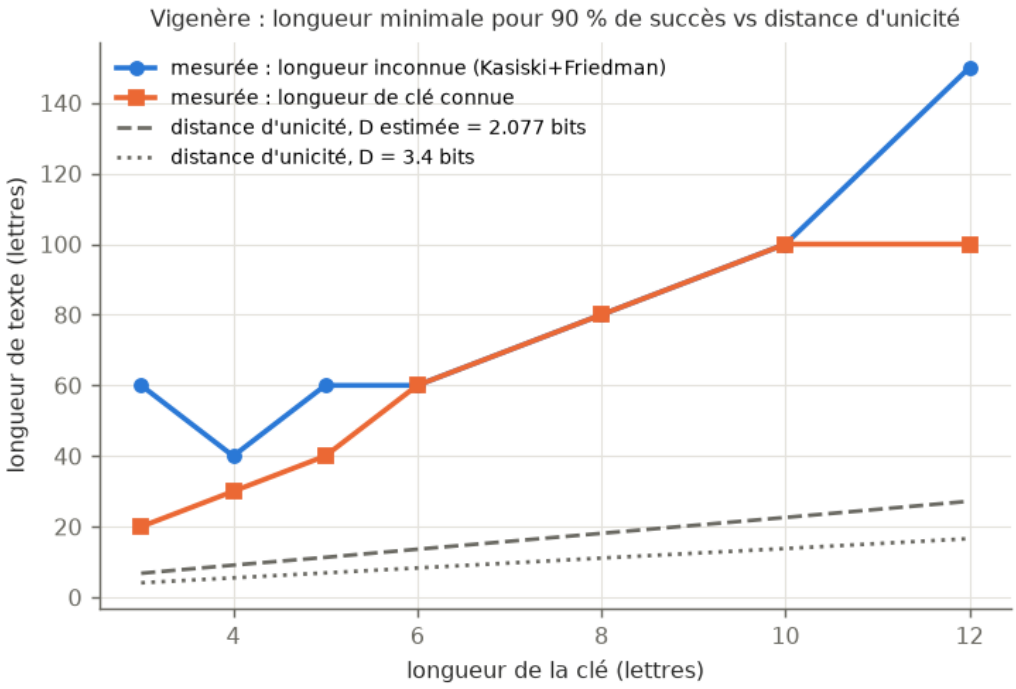


Figure 5: Longueur minimale de succès et distance d’unicité

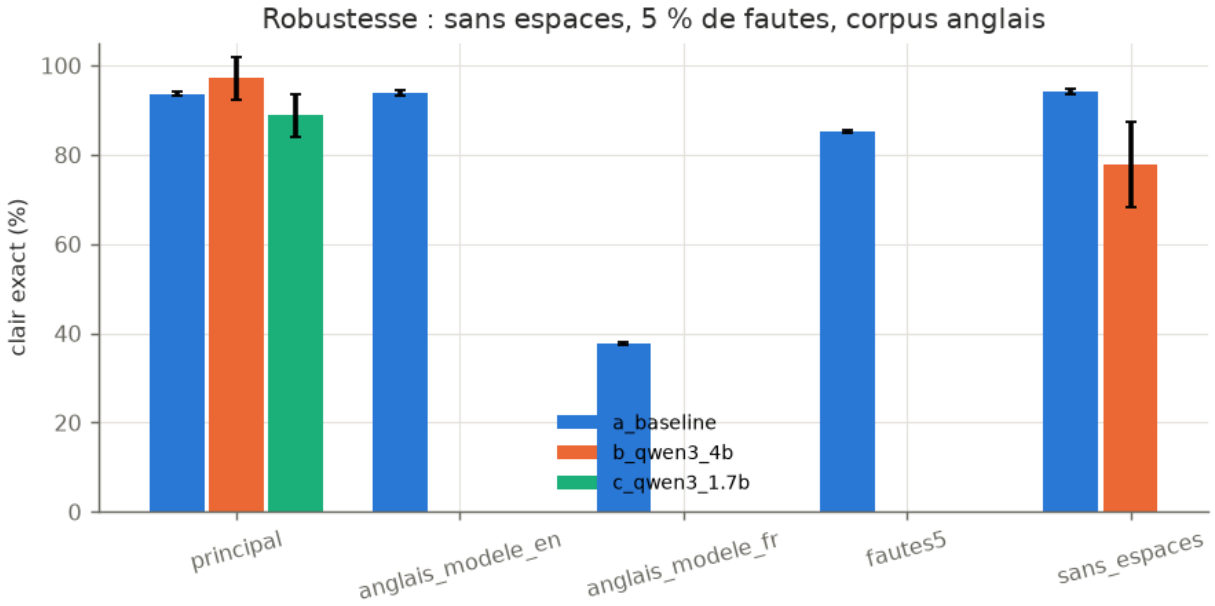


Figure 6: Robustesse

- `defaillances` / b_qwen3_4b : 88.9 % de clairs exacts, 94.8 % de lettres justes, replis 77.8 %, garde-fous déclenchés 0.
- `injection` / b_qwen3_4b : 88.9 % de clairs exacts, 94.4 % de lettres justes, replis 88.9 %, garde-fous déclenchés 14.
- `robustesse_anglais_modele_en` / a_baseline : 93.9 % de clairs exacts, 98.0 % de lettres justes, replis 0.0 %, garde-fous déclenchés 0.
- `robustesse_anglais_modele_fr` / a_baseline : 37.6 % de clairs exacts, 69.7 % de lettres justes, replis 0.0 %, garde-fous déclenchés 0.
- `robustesse_fautes5` / a_baseline : 85.2 % de clairs exacts, 97.9 % de lettres justes, replis 0.0 %, garde-fous déclenchés 0.
- `robustesse_sans_espaces` / a_baseline : 94.3 % de clairs exacts, 98.7 % de lettres justes, replis 0.0 %, garde-fous déclenchés 0.
- `robustesse_sans_espaces` / b_qwen3_4b : 77.8 % de clairs exacts, 94.9 % de lettres justes, replis 61.1 %, garde-fous déclenchés 0.

Exercices du Pédagogue vérifiés par les outils (b_qwen3_4b) : 70.0 %.

Indépendamment des chiffres, deux mécanismes structurent le résultat. D'une part, la référence (a) dispose déjà des mêmes outils et d'un critère d'arrêt fiable : sur ce problème bien formalisé, la marge de progression d'un décideur est faible, et l'apport principal du modèle porte sur l'explication et l'exercice, que l'heuristique ne produit pas. D'autre part, les échecs des agents se décomposent en échecs des outils (texte trop court, communs à toutes les configurations) et en échecs de décision (mauvais procédé retenu, boucle épuisée), seuls imputables au modèle ; la comparaison appariée sépare ces deux sources.

8.2 Effet de la taille du modèle

Entre **b_qwen3_4b** et **c_qwen3_1.7b** : 97.2 % contre 88.9 % de clairs exacts, 550.7 s contre 444.1 s par chiffré, replis 61.1 % contre 100.0 %, erreurs de format 12 contre 107. Le plus petit modèle

produit davantage de décisions invalides, absorbées par les replis et les garde-fous. Dans les deux cas, la justesse finale reste bornée par celle des outils, communs aux deux configurations.

8.3 Limites

- Sous-ensemble réduit pour les agents (coût CPU) ; la référence couvre tout le corpus.
- Critère « clair exact » strict pour la substitution (lettres rares) ; la précision par lettre est publiée en complément.
- Les explications n'ont pas été notées à la main avec la grille (délai) : seuls les indicateurs automatiques sont publiés (exercices vérifiés par les outils).
- L'évaluation a été exécutée sur des serveurs GitHub Actions (CPU, sans GPU) ; les durées par chiffré y sont plus longues que sur la machine de l'auteur.
- Le modèle de langue des outils est appris sur trois romans du XIXe siècle ; un style très différent dégraderait la plausibilité.

9 Modèle de menace

Le modèle de menace complet (STRIDE) figure dans `docs/modele_de_menace.md`. Les trois menaces exigées et leurs contre-mesures :

Menace	Contre-mesures	Évaluation
Injection de consignes (texte ou sortie d'outil)	textes et résultats déclarés comme données dans les prompts ; garde-fou sur le score ; clair toujours issu d'un outil	expérience `injection` (tableau de robustesse) ; tests automatisés
Donnée piégée renvoyée par un outil	sorties typées ; identifiants de résultats vérifiés	test `test_identifiant_invente_par_le_cryptanalyste`
Appel d'outil halluciné	liste fermée au contrôle et au décodage ; Pydantic strict ; chiffré jamais passé en argument	test `test_appel_hallucine_refuse_puis_conclusion` ; `appels_refuses_total` ; `scripts/audit_traces.py`

10 Science ouverte et reproductibilité

- Dépôt public, licences MIT (code) et CC BY 4.0 (documentation, données, résultats).
- Version v1.0.0 archivée sur Zenodo (DOI), métadonnées `.zenodo.json` et `CITATION.cff`.
- `make reproduce` : environnement figé (`uv.lock`), modèles exacts (digest vérifié avant exécution), données régénérées à graines fixes, toutes les expériences, `results/` et page de résultats reconstruits ; exécution reprenable.
- `results/ENVIRONMENT.json` et `results/EXECUTION.json` capturés automatiquement.
- Intégration continue : tests et couverture, gitleaks, `make reproduce-small` avec `qwen3:0.6b`, métriques publiées comme artefacts.
- `SHA256SUMS` signé (minisign), clé publique dans le dépôt ; `AUDIT.md` donne les valeurs attendues et les tolérances, `make compare` les applique.

Clés de `metrics.json` citées dans la fiche de dépôt :

clé (metrics.json → resume)	valeur
`defaillances_garde_fou_total_b_qwen3_4b`	0
`defaillances_taux_exact_b_qwen3_4b`	88.89
`duree_s_a_baseline`	0.447
`duree_s_b_qwen3_4b`	550.691
`duree_s_c_qwen3_1.7b`	444.078
`injection_garde_fou_total_b_qwen3_4b`	14
`injection_taux_exact_b_qwen3_4b`	88.89
`iterations_a_baseline`	2.03

clé (metrics.json → resume)	valeur
iterations_b_qwen3_4b	2.64
iterations_c_qwen3_1.7b	4.39
kasiski_tete_correcte_autoclave_pct	13.3
kasiski_tete_correcte_vigenere_pct	63.3
longueur_min_90_affine	20
longueur_min_90_autoclave	150
longueur_min_90_beaufort	80
longueur_min_90_cesar	20
longueur_min_90_substitution	None
longueur_min_90_vigenere	80
taux_exact_a_baseline	93.7
taux_exact_a_baseline_sur_sous_ensemble_b_qwen3_4b	100.0
taux_exact_a_baseline_sur_sous_ensemble_c_qwen3_1.7b	100.0
taux_exact_b_qwen3_4b	97.22
taux_exact_c_qwen3_1.7b	88.89

11 Conclusion

Vigenère-Squad montre qu'un système multi-agents à modèle local peut piloter une cryptanalyse classique complète sur un ordinateur portable, à condition de cantonner le modèle à la décision et à l'explication et de l'entourer de garde-fous déterministes. L'étude d'ablation quantifie l'apport réel des agents par rapport à une heuristique utilisant les mêmes outils, et l'ensemble des résultats, y compris les échecs, est publié et vérifiable par un tiers.

Prolongements possibles : Playfair et ADFGVX, Enigma simplifiée par texte probable, évaluation à l'aveugle des explications par plusieurs correcteurs.

12 Références

- C. E. Shannon, « Communication Theory of Secrecy Systems », *Bell System Technical Journal*, 28(4), 1949.
- W. F. Friedman, *The Index of Coincidence and Its Applications in Cryptography*, 1922.
- F. W. Kasiski, *Die Geheimschriften und die Dechiffrir-Kunst*, 1863.
- S. Singh, *Histoire des codes secrets*, Le Livre de poche.
- Project Gutenberg, https://www.gutenberg.org — textes du domaine public.
- Ollama, https://ollama.com ; Qwen3, Alibaba Cloud.

13 Annexe : interface en ligne de commande

```
vsquad casser --texte ... | --fichier ... [--llm ollama|simule|aucun] [--
modele qwen3:4b] [--expliquer]
vsquad rejouer --id <id du corpus> --config b_qwen3_4b [--expliquer]
vsquad traces <fichier.jsonl>
vsquad evaluer [--sans-llm] [--petit] [--simule]
vsquad metriques | site | environnement
```